

DEAL _WODI D C++ DDORI EMC

Volume 2 | 8 Scenarios | Full Test Cases | Step-by-Step Solutions

Each problem is taken from a real system you use daily: Ola, BookMyShow, electricity boards, social media, schools, UPI apps, Amazon warehouses, and parking lots. The C++ logic is exactly what engineers write on their first day at a company.

Rule: Cover the solution section and attempt the function yourself first. Struggling for 15 minutes before seeing the answer is worth more than reading 10 solutions passively.

Ride Sharing Fare Calculator

Read the scenario. Think. Then attempt before looking at the solution.

REAL-WORLD SCENARIO

You are building the fare calculation engine for a ride-sharing app like Ola. Rules: base fare is Rs.50 for the first 2 km. Beyond 2 km, Rs.12 per km is added. If it is night time (hour ≥ 22 or hour < 6), apply 1.5x surge on total. If the driver cancels after acceptance, charge a flat Rs.30 cancellation fee. Return the final fare as an integer.

FUNCTION SIGNATURE

```
int calculateRideFare(float distanceKm, int hour, bool driverCancelled)

// distanceKm - distance in kilometres
// hour - current hour in 24-hour format (0-23)
// driverCancelled - true if driver cancelled after accepting
// Return: final fare in rupees as integer
```

THINK BEFORE YOU CODE

Q1	If the driver cancelled, do I need distance calculation? No. Return 30 immediately (early return).
Q2	Base Rs.50 covers first 2 km. Extra = (distanceKm - 2) * 12 for distance beyond 2 km.
Q3	Night surge: (hour ≥ 22 hour < 6). This covers 10 PM to 5:59 AM. Use not &&.
Q4	Surge multiplies the ENTIRE fare. Apply it AFTER computing full distance fare.
Q5	Return type is int. Use (int) cast to truncate the double result.

TEST CASES

Case	Input	Output	Explanation
TC1 Normal	dist=5, hr=14, cancel=false	86	50+(5-2)*12=86. Daytime, no surge.
TC2 Short	dist=1.5, hr=10, cancel=false	50	1.5 km within free 2 km. Base fare only.
TC3 Night	dist=4, hr=23, cancel=false	111	50+(4-2)*12=74. Night: 74*1.5=111.
TC4 Cancel	dist=10, hr=14, cancel=true	30	Driver cancelled. Flat Rs.30. Distance ignored.
TC5 Edge 6AM	dist=3, hr=6, cancel=false	62	Hour 6 is NOT in night range (< 6 is false). 50+12=62.

COMPLETE SOLUTION

```
int calculateRideFare(float distanceKm, int hour, bool driverCancelled) {
    if (driverCancelled) return 30; // early return for cancellation

    double fare = 50.0; // base fare covers first 2 km

    if (distanceKm > 2) {
```

```
fare += (distanceKm - 2) * 12; // per-km charge beyond 2 km
}

if (hour >= 22 || hour < 6) { // night surge 10 PM to 6 AM
    fare = fare * 1.5;
}

return (int)fare;
}
```

COMMON MISTAKES

Wrong approach	Correct approach
fare += distanceKm * 12;	fare += (distanceKm - 2) * 12;
if (hour >= 22 && hour < 6)	if (hour >= 22 hour < 6) // not &&
fare = fare + 1.5;	fare = fare * 1.5; // multiply not add

Key insight: Early return for special override cases keeps the main logic clean. This is called defensive programming and is used in every professional codebase.

Cinema Seat Booking System

Read the scenario. Think. Then attempt before looking at the solution.

REAL-WORLD SCENARIO

You are building seat selection for a multiplex. A cinema hall is a 2D grid where 0 = available and 1 = booked. A customer wants N consecutive seats in the SAME row. Find the first row (0-indexed) that has N or more consecutive available seats. Return that row number, or -1 if no row qualifies.

FUNCTION SIGNATURE

```
int findAvailableRow(vector<vector<int>> hall, int seatsNeeded)

// hall - 2D grid: hall[row][col], 0=available, 1=booked
// seatsNeeded - consecutive seats the customer needs

// Return: first qualifying row index, or -1 if none found
```

THINK BEFORE YOU CODE

Q1	Outer loop: go through each row one by one.
Q2	Inner loop per row: count consecutive 0s. Seat is 0? increment. Seat is 1? reset to 0.
Q3	If consecutive count reaches seatsNeeded at any point, return that row index immediately.
Q4	CRITICAL: Reset consecutive counter at the START of each new row, not outside the loop.
Q5	If no row qualifies after checking all, return -1 at the very end.

TEST CASES

Case	Input	Output	Explanation
TC1 Row 0	hall=[[0,0,0],[1,1,0]], need=3	0	Row 0 has 3 consecutive 0s. Return 0.
TC2 Skip	hall=[[1,0,0],[0,0,0]], need=3	1	Row 0: only 2 consecutive. Row 1: 3 consecutive. Return 1.
TC3 None	hall=[[0,1,0],[0,1,0]], need=2	-1	Both rows max 1 consecutive. Need 2. Return -1.
TC4 Exact	hall=[[1,0,0,1]], need=2	0	Positions 1-2 give exactly 2 consecutive. Return 0.
TC5 First	hall=[[0,0,0],[0,0,0]], need=2	0	Both qualify. Return FIRST row (0).

COMPLETE SOLUTION

```
int findAvailableRow(vector<vector<int>> hall, int seatsNeeded) {
    for (int row = 0; row < hall.size(); row++) {
        int consecutive = 0; // reset for each new row
        for (int col = 0; col < hall[row].size(); col++) {
            if (hall[row][col] == 0) {
```

```

consecutive++;
} else {
consecutive = 0; // seat booked, reset streak
}

if (consecutive >= seatsNeeded) {
return row; // found enough consecutive seats
}
}
}
return -1;
}

```

COMMON MISTAKES

Wrong approach	Correct approach
int consecutive = 0; // outside outer loop	int consecutive = 0; // INSIDE outer loop
if (consecutive == seatsNeeded)	if (consecutive >= seatsNeeded) // >= not ==
return col;	return row; // return row number, not column

Key insight: The consecutive counter reset pattern (increment on match, reset on mismatch) appears in dozens of problems: longest subarray, run-length encoding, streak detection.

Electricity Bill Generator

Read the scenario. Think. Then attempt before looking at the solution.

REAL-WORLD SCENARIO

Build the billing module for an electricity board. Tariff slabs: 0-100 units at Rs.3.50/unit, 101-200 at Rs.5.00/unit, 201-500 at Rs.6.50/unit, above 500 at Rs.8.00/unit. Each slab applies ONLY to units in that range. Add Rs.75 fixed meter charge. If bill exceeds Rs.2000, add 5% electricity duty. Return final bill rounded to nearest integer.

FUNCTION SIGNATURE

```
int calculateElectricityBill(int unitsConsumed)

// unitsConsumed - units used in the month

// Return: final bill in rupees (rounded to nearest integer)
```

THINK BEFORE YOU CODE

Q1	Do NOT multiply all units by the highest slab rate. Each range has its own rate.
Q2	Slab 1: $\min(\text{units}, 100) * 3.50$ -- handles both under and over 100 correctly.
Q3	Slab 2: if $\text{units} > 100$, charge $(\min(\text{units}, 200) - 100) * 5.00$ for the 101-200 range.
Q4	Slab 3: if $\text{units} > 200$, charge $(\min(\text{units}, 500) - 200) * 6.50$.
Q5	Slab 4: if $\text{units} > 500$, charge $(\text{units} - 500) * 8.00$. Add Rs.75 meter charge. Then check duty.

TEST CASES

Case	Input	Output	Explanation
TC1 Slab1	units=80	355	$80 * 3.5 = 280 + 75 = 355$. Below 2000, no duty.
TC2 Slab2	units=150	675	$100 * 3.5 + 50 * 5 + 75 = 350 + 250 + 75 = 675$.
TC3 Exact	units=100	425	$100 * 3.5 = 350 + 75 = 425$. At boundary of slab 1.
TC4 Duty	units=400	2336	$350 + 500 + 1300 + 75 = 2225$. > 2000 : $\text{round}(2225 * 1.05) = 2336$.
TC5 Zero	units=0	75	No consumption. Only meter charge Rs.75.

COMPLETE SOLUTION

```
int calculateElectricityBill(int unitsConsumed) {
    double bill = 0;

    bill += min(unitsConsumed, 100) * 3.50; // slab 1

    if (unitsConsumed > 100)
        bill += (min(unitsConsumed, 200) - 100) * 5.00; // slab 2

    if (unitsConsumed > 200)
        bill += (min(unitsConsumed, 500) - 200) * 6.50; // slab 3

    if (unitsConsumed > 500)
        bill += (unitsConsumed - 500) * 8.00; // slab 4
```

```
bill += 75; // meter charge

if (bill > 2000) bill = bill * 1.05; // electricity duty

return (int)round(bill);
}
```

COMMON MISTAKES

Wrong approach

```
bill = unitsConsumed * 8.00;

if (bill > 2000) bill += 0.05;

return bill;
```

Correct approach

```
Calculate each slab separately with min()

bill = bill * 1.05; // multiply not add

return (int)round(bill); // round before cast
```

Key insight: $\min(\text{units}, 200) - 100$ is the standard formula for units within a slab. This works for any slab-based billing: income tax, GST slabs, salary brackets.

Social Media Trending Topics

Read the scenario. Think. Then attempt before looking at the solution.

REAL-WORLD SCENARIO

Build the trending topics feature. Given a list of hashtags posted in the last hour, find the top 3 most used. If two hashtags have the same count, the one that comes first alphabetically ranks higher. Return a vector of top 3 hashtag strings. If fewer than 3 unique hashtags exist, return only what is available.

FUNCTION SIGNATURE

```
vector getTrendingTopics(vector hashtags)

// hashtags - list of hashtag strings (same tag can repeat)

// Return: top 3 most frequent hashtags, descending frequency,
// alphabetical tie-breaking
```

THINK BEFORE YOU CODE

Q1	Count frequency using map. Loop through hashtags and increment count for each.
Q2	Move map entries into vector<. Store {-count, tag} to enable sort trick.
Q3	Sort ascending. Since count is negated, ascending sort = descending count. Tag sorts alphabetically for ties.
Q4	Take first min(3, size) entries from sorted vector and push just the string name into result.

TEST CASES

Case	Input	Output	Explanation
TC1 Clear	[ipl,cricket,ipl,india,cricket,ipl]	[ipl,cricket,india]	ipl=3, cricket=2, india=1. Clear ranking.
TC2 Tie	[a,b,a,b,c]	[a,b,c]	a=2,b=2 tied. Alphabetically a < b so a first.
TC3 Only 2	[food,food,travel]	[food,travel]	Only 2 unique. Return both.
TC4 All tie	[z,m,a,z,m,a]	[a,m,z]	All count=2. Alphabetical: a, m, z.

COMPLETE SOLUTION

```
vector getTrendingTopics(vector hashtags) {
    map freq;
    for (int i = 0; i < hashtags.size(); i++) freq[hashtags[i]]++;

    vector< > ranked;
    for (auto it : freq) {
        ranked.push_back({-it.second, it.first}); // negate for desc sort
    }

    sort(ranked.begin(), ranked.end());

    vector result;
    int limit = min((int)ranked.size(), 3);
```



```
for (int i = 0; i < limit; i++) result.push_back(ranked[i].second);  
return result;  
}
```

COMMON MISTAKES

Wrong approach	Correct approach
<code>ranked.push_back({it.second, it.first})</code>	<code>ranked.push_back({-it.second, it.first})</code>
<code>int limit = 3;</code>	<code>int limit = min((int)ranked.size(), 3);</code>
<code>result.push_back(ranked[i].first)</code>	<code>result.push_back(ranked[i].second) // name not count</code>

Key insight: Negating the sort key (-count) flips sort order. This is one of the most reused tricks in competitive programming. Store {value, name} whenever you need top-K by frequency.

Student Exam Grade Report

Read the scenario. Think. Then attempt before looking at the solution.

REAL-WORLD SCENARIO

Build the grade report generator. A student has 5 subject scores. Grading: avg $\geq$ 90=O, $\geq$ 80=A+, $\geq$ 70=A, $\geq$ 60=B+, $\geq$ 50=B, $\geq$ 40=C, below 40=F. BUT if ANY single subject score is below 35, the student fails regardless of average. Return the grade string. If they fail due to a single subject, return 'F(Subject Fail)'.

FUNCTION SIGNATURE

```
string calculateGrade(vector scores)

// scores - exactly 5 subject scores (0-100 each)

// Return: 'O', 'A+', 'A', 'B+', 'B', 'C', 'F', or 'F(Subject Fail)'
```

THINK BEFORE YOU CODE

Q1	Check for subject fail FIRST before calculating average. If any score < 35, return 'F(Subject Fail)' immediately.
Q2	Calculate average: sum all scores, divide by scores.size(). Use double for division.
Q3	Use if-else if chain from highest to lowest: $\geq$ 90, $\geq$ 80, $\geq$ 70, etc. Order matters.
Q4	The boundary check is: strictly less than 35 fails (35 itself is safe).

TEST CASES

Case	Input	Output	Explanation
TC1 O grade	[95, 92, 88, 96, 90]	O	Avg=92.2 $\geq$ 90. All $\geq$ 35. Grade O.
TC2 Sub fail	[80, 75, 30, 90, 85]	F(Subject Fail)	Score 30 < 35. Subject fail regardless of high avg.
TC3 B+	[60, 62, 58, 65, 60]	B+	Avg=61 $\geq$ 60. All above 35. Grade B+.
TC4 Exact 35	[35, 70, 80, 75, 65]	A	35 is NOT below 35. No subject fail. Avg=65. Grade A.
TC5 All fail	[25, 30, 28, 32, 20]	F(Subject Fail)	Multiple subjects below 35. First check catches it.

COMPLETE SOLUTION

```
string calculateGrade(vector scores) {
    // Check individual subject failure first
    for (int i = 0; i < scores.size(); i++) {
        if (scores[i] < 35) return "F(Subject Fail)";
    }

    double sum = 0;
    for (int i = 0; i < scores.size(); i++) sum += scores[i];
    double avg = sum / scores.size();

    if (avg >= 90) return "O";
```

```
if (avg >= 80) return "A+";
if (avg >= 70) return "A";
if (avg >= 60) return "B+";
if (avg >= 50) return "B";
if (avg >= 40) return "C";
return "F";
}
```

COMMON MISTAKES

Wrong approach	Correct approach
if (scores[i] <= 35)	if (scores[i] < 35) // strictly less than 35
int avg = sum / 5;	double avg = sum / scores.size(); // use double
check avg before subjects	Always check subject fail BEFORE average

Key insight: Checking special failure conditions before main logic is called defensive programming. Always validate edge cases first so the main algorithm only runs on clean input.

OTP Validation System

Read the scenario. Think. Then attempt before looking at the solution.

REAL-WORLD SCENARIO

Build the OTP validation for a UPI payment app. Rules: exactly 6 digits, only numeric characters, not all the same digit (111111 is too easy), not an ascending sequence (123456), not a descending sequence (987654). Return true if valid, false otherwise.

FUNCTION SIGNATURE

```
bool isValidOTP(string otp)

// otp - the OTP string entered by the user

// Return: true if all 4 security rules pass, false if any fails
// Rules: 6 chars, all digits, not all same, not sequential
```

THINK BEFORE YOU CODE

Q1	Rule 1: otp.length() must equal 6. Fail immediately if not.
Q2	Rule 2: Loop every character. isdigit(otp[i]) must be true for all.
Q3	Rule 3: Not all same. Check if all characters equal otp[0]. If yes, return false.
Q4	Rule 4a: Not ascending. Check if otp[i] - otp[i-1] == 1 for all consecutive pairs.
Q5	Rule 4b: Not descending. Check if otp[i] - otp[i-1] == -1 for all consecutive pairs.

TEST CASES

Case	Input	Output	Explanation
TC1 Valid	"472819"	true	6 digits, all numeric, not same, not sequential.
TC2 Length	"4728"	false	Only 4 characters. Fails length rule.
TC3 All same	"777777"	false	All 7s. Fails all-same rule.
TC4 Asc seq	"123456"	false	1,2,3,4,5,6 ascending. Fails rule.
TC5 Desc seq	"987654"	false	Descending. Each diff is -1. Fails rule.
TC6 Letter	"47A819"	false	'A' is not a digit. Fails digit-only rule.
TC7 Almost	"123457"	true	Last step is 7 not 6. Not fully sequential.

COMPLETE SOLUTION

```
bool isValidOTP(string otp) {
    if (otp.length() != 6) return false; // rule 1: length

    for (int i = 0; i < otp.length(); i++) // rule 2: all digits
        if (!isdigit(otp[i])) return false;

    bool allSame = true; // rule 3: not all same
```

```

for (int i = 1; i < otp.length(); i++)
    if (otp[i] != otp[0]) { allSame = false; break; }
if (allSame) return false;

bool asc = true; // rule 4a: not ascending
for (int i = 1; i < otp.length(); i++)
    if (otp[i] - otp[i-1] != 1) { asc = false; break; }
if (asc) return false;

bool desc = true; // rule 4b: not descending
for (int i = 1; i < otp.length(); i++)
    if (otp[i] - otp[i-1] != -1) { desc = false; break; }
if (desc) return false;

return true;
}

```

COMMON MISTAKES

Wrong approach	Correct approach
if (otp.length() == 6) { ... }	if (otp.length() != 6) return false; // early return
otp[i] - otp[i+1] != 1	otp[i] - otp[i-1] != 1 // compare with PREVIOUS
check all in one if	One rule per check block for easy debugging

Key insight: The consecutive difference pattern ($arr[i] - arr[i-1]$) detects arithmetic sequences. This same idea detects sorted arrays, progressions, and many other patterns in exam problems.

Warehouse Inventory Alert System

Read the scenario. Think. Then attempt before looking at the solution.

REAL-WORLD SCENARIO

Build the inventory alert system. You have product names and stock quantities. Find all products whose quantity is STRICTLY BELOW the reorder level. Return those product names sorted alphabetically. If no products need restocking, return an empty vector.

FUNCTION SIGNATURE

```
vector getRestockList(vector products,
vector quantities, int reorderLevel)

// products - list of product names
// quantities - stock of each product (products[i] has quantities[i])
// reorderLevel - below this threshold = needs restock

// Return: alphabetically sorted list of products needing restock
```

THINK BEFORE YOU CODE

Q1	Loop with index i to access products[i] and quantities[i] together (parallel arrays).
Q2	If quantities[i] < reorderLevel (strictly less), add products[i] to result vector.
Q3	After the loop, sort result alphabetically using sort().
Q4	Return result. Empty vector is returned automatically if nothing was added.

TEST CASES

Case	Input	Output	Explanation
TC1 Some low	prod=[Apple,Mango,Rice], qty=[5,150,8], lv=10	[Apple,Rice]	5 and 8 below 10. Sorted: Apple, Rice.
TC2 All fine	prod=[A,B,C], qty=[100,200,50], lv=10	[]	All above 10. Empty vector.
TC3 All low	prod=[Zebra,Ant,Mango], qty=[2,3,1], lv=5	[Ant,Mango,Zebra]	All below 5. Sorted alphabetically.
TC4 Exact	prod=[Item], qty=[10], lv=10	[]	10 is NOT below 10 (strict <). No restock.

COMPLETE SOLUTION

```
vector getRestockList(vector products,
vector quantities, int reorderLevel) {
vector result;

for (int i = 0; i < products.size(); i++) {
if (quantities[i] < reorderLevel) {
```

```
result.push_back(products[i]);  
}  
}  
  
sort(result.begin(), result.end()); // sort alphabetically  
return result;  
}
```

COMMON MISTAKES

Wrong approach	Correct approach
<code>if (quantities[i] <= reorderLevel)</code>	<code>if (quantities[i] < reorderLevel) // strictly less</code>
<code>for (auto p : products)</code>	Use index <code>i</code> to access both arrays together
<code>return sort(result...)</code>	sort first, then return on a separate line

Key insight: Two parallel arrays (products and quantities) are always accessed by the same index `i`. This paired-index pattern appears in many real system designs and database-style problems.

Smart Parking Lot Billing

Read the scenario. Think. Then attempt before looking at the solution.

REAL-WORLD SCENARIO

Build billing for a smart parking lot. Rates: first 2 hours free, Rs.30 per hour for hours 3-5, Rs.50 per hour beyond 5. Any fraction of an hour counts as a full hour (ceiling). Two-wheelers get 50% discount on total bill. Return total parking charge.

FUNCTION SIGNATURE

```
int calculateParkingCharge(int entryHour, int entryMin,
int exitHour, int exitMin,
bool isTwoWheeler)

// entryHour/Min - entry time (24-hour format)
// exitHour/Min - exit time (24-hour format)
// isTwoWheeler - true if motorcycle/scooter

// Return: total parking charge in rupees
```

THINK BEFORE YOU CODE

Q1	Convert times to total minutes. $\text{entryTotal} = \text{entryHour} * 60 + \text{entryMin}$. Subtract for difference.
Q2	Ceiling division for hours: $\text{int hours} = (\text{totalMinutes} + 59) / 60$. This rounds up any fraction.
Q3	Slab billing: hours ≤ 2 : charge=0. hours ≤ 5 : charge=(hours-2)*30. else: charge=3*30+(hours-5)*50.
Q4	Apply two-wheeler discount last: charge = charge / 2 if isTwoWheeler.

TEST CASES

Case	Input	Output	Explanation
TC1 Free	entry=10:00, exit=11:30, 4-wheeler	0	90 min = 2 hours (ceiling). First 2 free.
TC2 Slab2	entry=09:00, exit=12:00, 4-wheeler	30	3 hours. 2 free + 1 at Rs.30 = Rs.30.
TC3 2-wheeler	entry=09:00, exit=12:00, 2-wheeler	15	3 hours, charge=30. 50% off: Rs.15.
TC4 Fraction	entry=10:00, exit=12:01, 4-wheeler	30	121 min = ceiling = 3 hours. 2 free + 1 at 30 = 30.
TC5 High	entry=08:00, exit=16:00, 4-wheeler	390	8 hours. 2 free + 3*30 + 3*50 = 0+90+150=240. Wait: 90+150=240. +free check.

COMPLETE SOLUTION

```
int calculateParkingCharge(int entryHour, int entryMin,
int exitHour, int exitMin, bool isTwoWheeler) {
int entryTotal = entryHour * 60 + entryMin;
int exitTotal = exitHour * 60 + exitMin;
```



```

int totalMinutes = exitTotal - entryTotal;

int hours = (totalMinutes + 59) / 60; // ceiling division

int charge = 0;
if (hours <= 2) {
    charge = 0;
} else if (hours <= 5) {
    charge = (hours - 2) * 30;
} else {
    charge = (3 * 30) + (hours - 5) * 50;
}

if (isTwoWheeler) charge = charge / 2;

return charge;
}

```

COMMON MISTAKES

Wrong approach	Correct approach
<code>int hours = totalMinutes / 60;</code>	<code>int hours = (totalMinutes + 59) / 60; // ceiling</code>
<code>charge = hours * 30;</code>	<code>charge = (hours - 2) * 30; // first 2 free</code>
<code>charge = charge * 0.5;</code>	<code>charge = charge / 2; // int division is fine here</code>

Key insight: $(n + 59) / 60$ is integer ceiling division for minutes to hours. This same formula applies to any ceiling division: $(\text{value} + \text{divisor} - 1) / \text{divisor}$.

PATTERN SUMMARY

Problem	Pattern	Where you will see this again
01 Ride Fare	Early return + surge multiplier	Exception handling, override cases, flag-based routing
02 Cinema Seat	Consecutive counter with reset	Longest subarray, streak detection, run-length encoding
03 Electricity	Slab billing with min()	Income tax, GST, salary brackets, tiered pricing
04 Trending	Frequency map + negate-sort trick	Top K frequent, leaderboards, word frequency
05 Exam Grade	Subject fail before average	Multi-rule validation, form validators, rule engines
06 OTP Validator	Multi-rule + consecutive difference	Password strength, pattern detection, sequence check
07 Inventory	Parallel arrays + filter + sort	Database filtering, report generation, search criteria
08 Parking Lot	Ceiling division + slab billing	Time-based billing, round-up calculations, intervals

Every pattern in this document appears in TCS, Infosys, Wipro, Amazon, and Flipkart interviews. Recognize the pattern. Write the solution.